

Team 7 - E-Waste Battery Extraction

CV-21 Part B: Inference Performance Optimisation & Validation

Experimental Method and Results

ONNX Runtime, ROS 2, Provider/Threading Experiments and Latency
Analysis

Ubuntu 24.04 on WSL2

Optimisation and validation report

September 2026

Contents

1	Purpose of Part B	2
2	Experimental Discipline	2
3	Runtime Environment	2
4	Experiment 1: Warm-up Behaviour	3
5	Experiment 2: Debug Image I/O	3
6	Experiment 3: ONNX Runtime Threading	4
7	Experiment 4: Execution Provider	5
8	Experiment 5: Preprocessing and Tensor Layout	6
9	Final Selected Configuration	7
10	Final End-to-End Benchmark	7
11	Evidence Preserved	9
12	Conclusion	10

1 Purpose of Part B

CV-21 is the **Inference Performance Profiling & Optimisation** task. Part A established an instrumented baseline and identified the dominant costs in the ROS 2 perception callback. Part B used that evidence to test practical optimisation options one factor at a time, reject changes that did not help, combine the useful changes, and validate the resulting configuration in the live Gazebo/ROS 2 pipeline.

The main optimisation areas investigated were:

- ONNX Runtime warm-up behaviour;
- per-frame debug image generation and PNG saving;
- ONNX Runtime CPU thread settings;
- execution provider configuration;
- preprocessing and tensor memory layout;
- final end-to-end latency and processing FPS.

2 Experimental Discipline

Part B was kept separate from the frozen Part A baseline. A dedicated working directory was created at:

`/mnt/e/team7_project_bundle/cv21_benchmark/part_b/`

Source snapshots were taken before significant changes so that the optimisation process remained reversible. The Part A benchmark was not overwritten.

For the main live comparisons, the same Gazebo scene, model, image topic, confidence threshold, NMS threshold, and 1280x960 overhead-camera stream were retained. The final control comparison used 220 measured frames per run, with the first 20 excluded, giving 200 steady-state frames per run.

3 Runtime Environment

The Part B environment was recorded as:

- Python 3.12.3;
- ONNX Runtime 1.29.0;
- available providers: AzureExecutionProvider and CPUExecutionProvider;
- no CUDAExecutionProvider;
- Intel Core i5-9300H, 4 physical cores / 8 logical CPUs;

- default ONNX Runtime execution mode: `ORT_SEQUENTIAL`;
- default graph optimisation level: `ORT_ENABLE_ALL`.

The original ROS node requested CUDA followed by CPU fallback. Since CUDA was not available, this produced a warning at startup and ultimately executed on CPU.

4 Experiment 1: Warm-up Behaviour

Three fresh ONNX Runtime sessions were created. In each session the first inference was treated as cold and the following runs as warmed-up.

Metric	Result
Mean first/cold inference	454.620 ms
Mean warmed inference	340.214 ms
Warm-up reduction	25.17%

Warm-up was therefore useful for startup behaviour. However, it was not treated as a claim of a 25.17% steady-state end-to-end improvement because Part A already excluded startup frames from its steady-state statistics.

5 Experiment 2: Debug Image I/O

The node originally performed three expensive debug operations on every frame:

1. save the raw image as PNG;
2. draw detection annotations;
3. save the annotated image as PNG.

Two configurable ROS parameters were added:

```
save_raw_images
save_annotated_images
```

Three live configurations were tested:

Run	Raw save	Annotation	Annotated save
A - control	ON	ON	ON
B - raw off	OFF	ON	ON
C - debug off	OFF	OFF	OFF

The main A-to-C comparison was:

Metric	Run A	Run C
Mean callback latency	527.174 ms	369.902 ms
Mean processing FPS	1.963	2.906
Raw PNG saving	61.891 ms	0.000 ms
Annotation drawing	57.939 ms	0.000 ms
Annotated PNG saving	66.159 ms	0.000 ms

This produced a 29.83% callback-latency reduction and a 48.06% increase in mean processing FPS. The directly removed debug path cost was 185.988 ms/frame.

Run B did not improve end-to-end mean latency even though raw saving reached 0 ms. In that run, CPU inference and the remaining annotation work were slower, demonstrating substantial run-to-run variability. This was useful evidence that a single stage should not be interpreted without considering the rest of the live system.

6 Experiment 3: ONNX Runtime Threading

A standalone probe tested the default ONNX Runtime thread management and explicit intra-operation thread counts of 1, 2, 4, and 8.

intra_op_num_threads	Mean inference
Default	74.378 ms
1	128.482 ms
2	107.521 ms
4	75.923 ms
8	168.730 ms

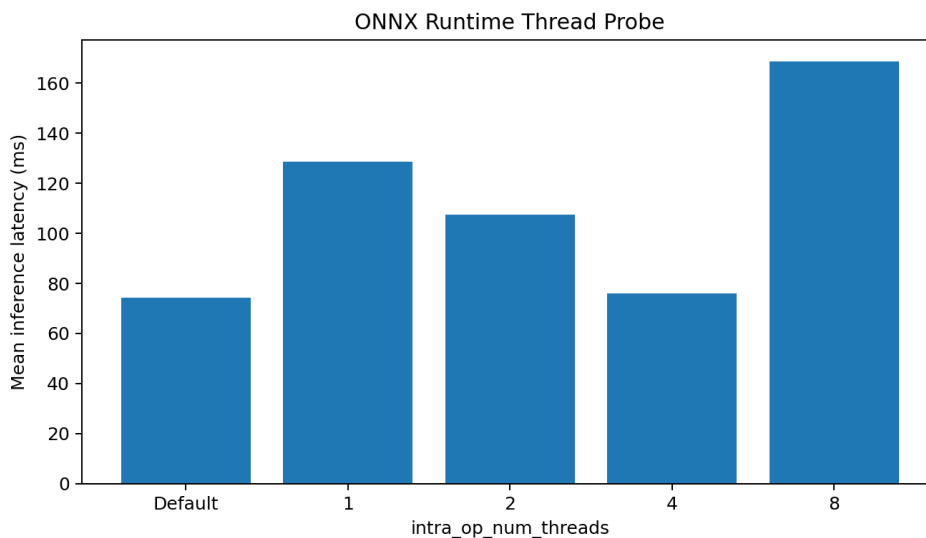


Figure 1: ONNX Runtime thread probe. The default setting produced the lowest mean inference latency.

The default configuration was retained. Explicitly forcing more threads did not help; in particular, 8 threads were much slower. This is consistent with scheduling, synchronisation, cache contention, and competition with Gazebo/ROS 2 for CPU resources.

7 Experiment 4: Execution Provider

An isolated provider probe compared CPU-only execution with AzureExecutionProvider plus CPU fallback.

Provider configuration	Mean inference	Median inference
CPU only	78.725 ms	78.478 ms
Azure + CPU fallback	70.596 ms	73.042 ms

Because the isolated result was promising, a live ROS/Gazebo comparison was then performed with debug image work disabled in both runs.

Metric	Run C: CPU	Run D: Azure + CPU
Mean callback latency	369.902 ms	324.119 ms
Mean processing FPS	2.906	3.249
Preprocessing	6.680 ms	6.561 ms
ONNX inference	322.415 ms	285.869 ms
Post-processing	17.731 ms	13.055 ms

The live provider change produced:

- 12.38% lower mean callback latency;
- 11.33% lower mean ONNX inference latency;
- 11.80% higher mean processing FPS.

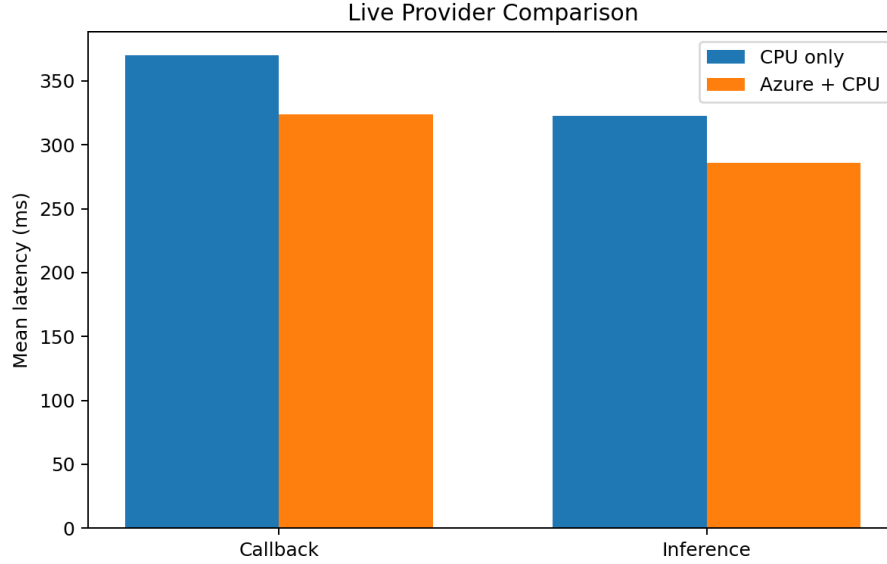


Figure 2: Live provider comparison with debug image processing disabled.

AzureExecutionProvider with CPU fallback was therefore selected for the final configuration.

8 Experiment 5: Preprocessing and Tensor Layout

The existing preprocessing performs letterboxing, BGR-to-RGB conversion, float32 normalisation, HWC-to-CHW transpose, and batch-dimension insertion. Inspection showed that the resulting baseline tensor was not C-contiguous.

Two alternatives were tested:

1. explicitly make the final tensor contiguous with `np.ascontiguousarray`;
2. replace the conversion path with `cv2.dnn.blobFromImage`.

All three inputs were numerically equivalent. The explicit contiguous tensor was exactly equivalent to the baseline within the tested precision, and `blobFromImage` differed only by approximately 5.96×10^{-8} .

The stronger combined measurement, preprocessing plus ONNX inference together, was:

Method	Mean combined latency
Original preprocessing	72.309 ms
Explicit contiguous tensor	78.025 ms
<code>blobFromImage</code>	75.669 ms

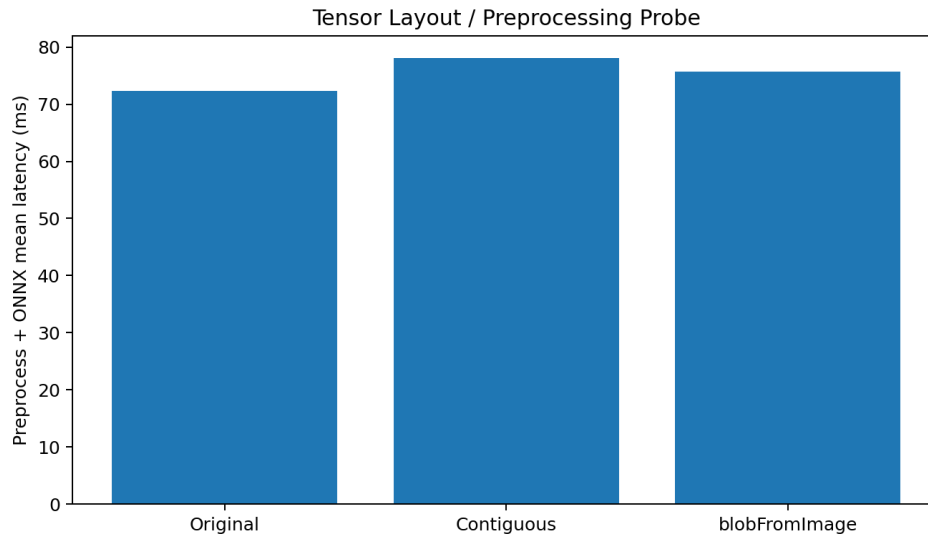


Figure 3: Combined preprocessing plus ONNX latency for the tested tensor-layout alternatives.

The original preprocessing was therefore retained. This is an important negative result: a memory-layout change that looked promising when preprocessing and `session.run()` were measured separately did not improve the combined operation.

9 Final Selected Configuration

The final CV-21 Part B configuration was:

- raw debug image saving: OFF by default;
- annotated image generation/saving: OFF by default;
- ONNX Runtime thread management: DEFAULT;
- provider mode: AzureExecutionProvider + CPUExecutionProvider fallback;
- preprocessing: ORIGINAL implementation retained;
- warm-up: documented as useful for startup behaviour.

The node remained configurable, so debug images can still be re-enabled when needed.

10 Final End-to-End Benchmark

The final comparison used 220 measured frames per run. The first 20 frames were excluded from each run, leaving 200 steady-state frames for both the control and final configurations.

Metric	Control A	Final E	Change
Mean callback latency	527.411 ms	382.364 ms	27.50% lower
Median callback latency	508.189 ms	346.844 ms	31.75% lower
P95 callback latency	679.593 ms	663.950 ms	2.30% lower
Mean processing FPS	1.962	2.887	47.13% higher

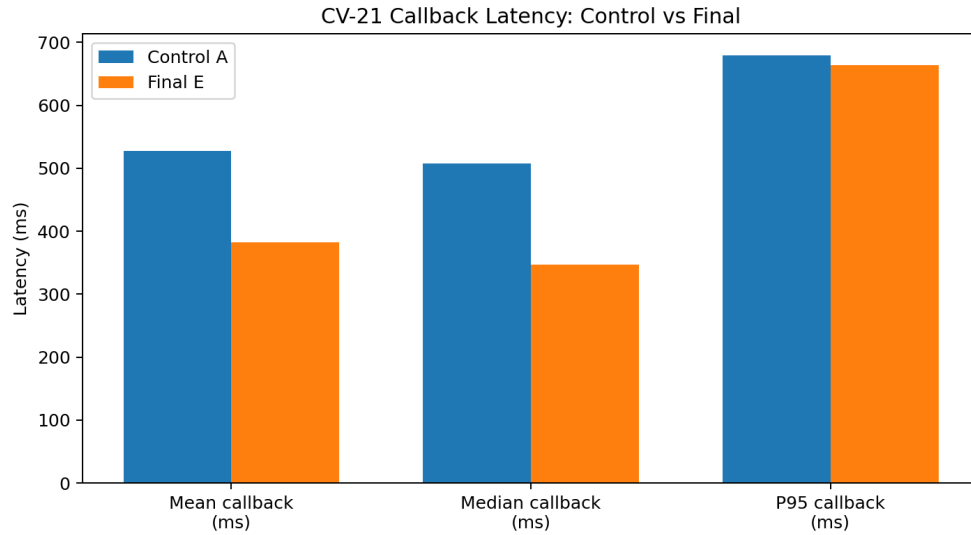


Figure 4: Final callback-latency comparison between the Part B control and selected configuration.

The final per-stage means were:

Stage	Control A	Final E
ROS image conversion	4.033 ms	3.702 ms
Raw PNG saving	61.969 ms	0.000 ms
Preprocessing	11.379 ms	11.328 ms
ONNX inference	293.871 ms	330.486 ms
Post-processing	18.345 ms	14.951 ms
Annotation drawing	57.821 ms	0.000 ms
Annotated PNG saving	66.200 ms	0.000 ms
Message construction	0.343 ms	0.274 ms
Metrics CSV writing	12.997 ms	20.460 ms
ROS publishing	0.334 ms	0.266 ms
Self-check	0.004 ms	0.003 ms

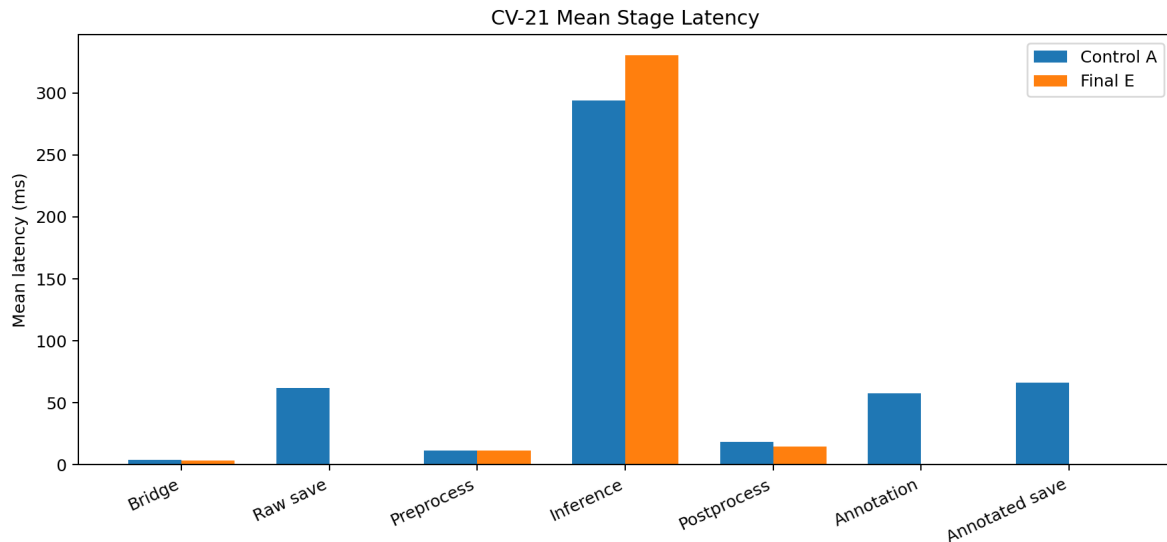


Figure 5: Mean stage latency for the final control and optimised runs.

The final run did not show a lower ONNX inference mean than Control A. Instead, inference increased from 293.871 ms to 330.486 ms in that particular run. This reflects the run-to-run CPU/runtime variability seen throughout Part B. Provider effectiveness should therefore be supported by the controlled Run C versus Run D experiment, where only the provider configuration changed and Azure + CPU reduced mean inference by 11.33%.

The final A-to-E comparison is used to demonstrate the complete selected configuration. Its strongest improvement came from removing non-essential per-frame debug image work, while the provider experiment separately demonstrated a runtime improvement under controlled conditions.

The final run also confirmed that zero raw and zero annotated image files were generated under the selected defaults.

11 Evidence Preserved

The Part B evidence structure contains:

```
cv21_benchmark/part_b/
  configs/
  logs/
  plots/
  results/
  source_snapshots/
```

Important logs include:

```
warmup_probe.txt
thread_probe.txt
provider_probe.txt
```

<code>preprocess_probe.txt</code> <code>layout_probe.txt</code>
--

Important final artefacts include the standardised benchmark CSVs, final comparison text, final plots, source snapshots, and hashes of the frozen final source files.

12 Conclusion

CV-21 Part B completed a full optimisation cycle rather than simply applying speculative changes. Each candidate was measured, useful changes were retained, and changes that did not improve the tested workload were rejected.

The final selected configuration achieved a **27.50% reduction in mean callback latency**, a **31.75% reduction in median callback latency**, and a **47.13% increase in mean processing FPS** relative to the controlled Part B baseline. At the same time, the work produced clear evidence about startup warm-up behaviour, ONNX Runtime thread settings, provider selection, debug I/O overhead, and tensor-layout alternatives.

Therefore, **CV-21 Part B is complete**.